

Contenerización de un servicio con Docker

Instrucciones

Lea este documento completo antes de repartir el trabajo dentro de su grupo. Esta tarea se resuelve en grupos de dos o tres personas, sin excepción; los grupos de tres entregan además el módulo adicional descrito más adelante, que no otorga puntos extra. Esta tarea no contempla defensa, de modo que todo elemento que deba evaluarse ha de quedar demostrado en el repositorio y en el video. Las fechas oficiales son las publicadas en Discord.

1 Descripción

El objetivo de esta tarea es la contenerización de un servicio propio y su orquestación junto a una base de datos PostgreSQL mediante Docker y Docker Compose. El grupo desarrolla un servicio HTTP, escribe su Dockerfile y el docker-compose.yml correspondiente, y justifica por escrito cada decisión de construcción y orquestación.

El alcance es reducido de forma deliberada. La evaluación no se centra en la elaboración del código, sino en su reproducibilidad, entendida como la capacidad de un evaluador ajeno al proyecto para clonar el repositorio, ejecutar un único comando y poner el sistema en funcionamiento. Lo que se evalúa son las decisiones de contenerización que la tarea ejercita, a saber, la selección de la imagen base, el contenido de la imagen, la comunicación entre servicios, la persistencia y la gestión de la configuración.

2 Núcleo obligatorio

El núcleo es obligatorio para grupos de dos y de tres personas por igual, y consta de los siguientes seis puntos:

1. **Un servicio HTTP propio.** El grupo desarrolla un servicio, en el lenguaje que prefiera, que exponga al menos tres rutas: una de salud (/health o equivalente) que responda sin acceder a la base de datos, una de escritura sobre PostgreSQL y una de lectura de lo registrado. El dominio es libre, pero el servicio debe mantener estado real en la base; una ruta que devuelva únicamente texto fijo no cumple el requisito.
2. **Un Dockerfile propio.** Debe partir de una imagen base oficial, instalar únicamente lo necesario, copiar el código del servicio, declarar el puerto y definir el comando de arranque. El README documenta la elección de la imagen base y las medidas adoptadas para excluir de la imagen los archivos que no le corresponden (dependencias del entorno de desarrollo, historial de Git, artefactos de compilación).

3. **Un `docker-compose.yml` que levante el sistema completo.** Tres servicios: la aplicación, PostgreSQL a partir de su imagen oficial y un proveedor de identidad externo (Keycloak) a partir de la suya. La aplicación debe alcanzar tanto la base como el proveedor *por su nombre de servicio* en la red que Compose crea, y no por una dirección IP fija ni por `localhost`.
4. **Autenticación delegada en un proveedor externo.** La autenticación no se implementa a mano, sino que se delega en un servicio de identidad de terceros, **Keycloak**, que corre como un contenedor más del Compose. El grupo configura en Keycloak un *realm* y un *client* para la aplicación, y el servicio protege sus rutas de modificación exigiendo un token de acceso (JWT/OIDC) emitido por Keycloak, cuya firma valida contra las llaves públicas del proveedor. Las rutas de salud y disponibilidad quedan abiertas. Una solicitud sin token válido recibe 401, y una con token válido pero sin el rol requerido recibe 403. El grupo no escribe su propio inicio de sesión ni su propia firma de tokens.
5. **Persistencia con un volumen.** Los datos de PostgreSQL residen en un volumen declarado en el Compose. El comportamiento esperado, que será verificado, es que los datos escritos persistan tras destruir y volver a crear los contenedores.
6. **Healthchecks y arranque ordenado.** Los servicios declaran su `healthcheck`, y el servicio de la aplicación no debe darse por listo mientras sus dependencias no lo estén. La estrategia para resolver la dependencia de arranque (`depends_on` con condición, reintentos con espera en la aplicación, o ambos) queda a criterio del grupo, pero debe estar justificada.
7. **Configuración por variables de entorno, sin secretos en la imagen.** Credenciales, nombre de la base, host y puerto se inyectan como variables de entorno. La imagen no debe contener ninguna contraseña, ni en el `Dockerfile`, ni en el código, ni en un archivo copiado dentro. En el repositorio se versiona un `.env.example` con las claves y valores de ejemplo; el `.env` real queda fuera por `.gitignore`.
8. **Pruebas unitarias y de integración.** La entrega incluye pruebas automatizadas. Las *unitarias* verifican la lógica del servicio de forma aislada, sin depender de servicios externos, como la validación de los campos de la entidad. Las *de integración* ejercitan los endpoints contra la pila real que levanta Compose, e incluyen la persistencia en PostgreSQL y la aceptación o el rechazo de un token frente a Keycloak. Las pruebas se ejecutan con un comando documentado en el README, sin pasos manuales.

Además del código, el repositorio debe incluir un README `.md` en Markdown que permita a una persona ajena al grupo poner el sistema en marcha sin consultas adicionales. Debe cubrir los requisitos previos, el comando de arranque, el contrato de cada ruta (método, cuerpo y respuesta esperada), la comprobación de la persistencia y el apagado del sistema.

Requisitos funcionales mínimos del servicio

El dominio es libre, pero el *contrato* del servicio no. Para que la entrega sea evaluable y reproducible por un tercero, el servicio debe cumplir lo siguiente, con independencia de lo que modele.

1. **Una entidad con estado real.** Definan una entidad del dominio con al menos tres campos además del identificador (por ejemplo, para una reserva: nombre, fecha y cantidad de personas). Esa entidad vive en una tabla de PostgreSQL.
2. **Inicialización automática del esquema.** Al levantar el sistema con un solo comando, la tabla debe crearse sola si no existe, mediante un script de inicialización, una migración o código de arranque del servicio. El evaluador no carga esquemas a mano.

3. **Comprobación de vida (*liveness*).** GET /health responde 200 con un cuerpo JSON, *sin* tocar la base de datos. Indica que el proceso está vivo, y es la ruta en la que se apoya el healthcheck del contenedor de la aplicación.
4. **Comprobación de disponibilidad (*readiness*).** GET /ready verifica la conexión con PostgreSQL y responde 200 cuando la base acepta consultas o 503 cuando no. Distinguir esta ruta de la anterior es lo que permite no dar por lista la aplicación antes de que su dependencia lo esté.
5. **Crear.** Un POST a la ruta del recurso (p. ej. /reservas) recibe un JSON, valida los campos obligatorios, inserta la fila y responde 201 con el recurso creado, incluido su identificador. Ante un cuerpo inválido o incompleto responde 400, no un 500 ni una caída del proceso.
6. **Listar y filtrar.** Un GET sobre el recurso devuelve 200 con la lista de lo que se haya escrito, y admite al menos un **filtro por parámetro de consulta** (por ejemplo, /reservas?fecha=2026-08-20) que restringe el resultado. La lista refleja lo que *otro* cliente escribió, no un estado en memoria del proceso.
7. **Consultar por identificador.** GET /reservas/{id} devuelve 200 con el elemento, o 404 cuando no existe.
8. **Actualizar.** Un PUT o PATCH a /reservas/{id} modifica el recurso y responde 200 con su versión actualizada. Devuelve 404 si el recurso no existe y 400 ante un cuerpo inválido.
9. **Eliminar.** Un DELETE a /reservas/{id} borra el recurso y responde 204 sin cuerpo, o 404 si no existe.
10. **Rutas protegidas.** Crear, actualizar y eliminar exigen un token de acceso válido emitido por Keycloak; sin él responden 401, y con un token sin el rol requerido, 403. Las rutas de salud y disponibilidad quedan siempre abiertas. Que listar y consultar sean públicas o protegidas queda a criterio del grupo, que lo documenta.
11. **Manejo de errores.** El servicio no debe interrumpirse ante una entrada mal formada ni ante la consulta de un recurso inexistente; responde con el código de estado correspondiente y permanece operativo.

Estos requisitos no exigen un *framework* extenso, autenticación ni interfaz gráfica. Fijan el mínimo necesario para que la persistencia sea verificable y para que las entregas de distintos grupos resulten comparables.

Criterios de aceptación

Cuatro comprobaciones deciden si el núcleo está funcionando, y todas se ejecutarán tal cual sobre su repositorio recién clonado:

1. **Un solo comando.** Tras copiar el .env.example a .env, docker compose up debe dejar el sistema operativo y ambos servicios en estado saludable, sin pasos manuales intermedios, sin cargar esquemas a mano y sin editar archivos.
2. **El contrato se cumple.** /health responde sin tocar la base y /ready refleja el estado de la conexión; el recurso recorre su ciclo completo (crear, listar con filtro, consultar por identificador, actualizar y eliminar) con los códigos de estado correctos; una ruta de modificación sin token obtiene 401 y con un token de Keycloak válido responde con éxito; una entrada inválida obtiene 400 y un recurso inexistente, 404, sin que el servicio se caiga.
3. **Sobrevivir un reinicio.** Escribir datos por la ruta de escritura, ejecutar docker compose down (sin borrar volúmenes), volver a levantar y leer los mismos datos por la ruta de lectura.

4. **Las pruebas corren.** El comando de pruebas documentado en el README ejecuta las unitarias y las de integración, y todas pasan.

3 Regla de frontera

Se prohíbe delegar en una herramienta o en un tercero la construcción de aquello que constituye el objeto de aprendizaje. En esta tarea la frontera se ubica en los archivos de construcción y orquestación, de modo que **el Dockerfile y el docker-compose.yml deben ser escritos por el grupo**, que debe poder explicar la función de cada instrucción.

En consecuencia, no se acepta referenciar en el Compose una imagen ya construida que realice el trabajo del servicio sin un Dockerfile propio, ni generar el Dockerfile o el docker-compose.yml con una herramienta que los produzca automáticamente a partir del proyecto, ni entregar un archivo copiado de una fuente externa cuyo contenido el grupo no pueda justificar. Una instrucción que el grupo no sepa fundamentar no se contabiliza.

Sí se permite el uso de imágenes base oficiales del ecosistema (postgres, python:slim, node:slim, golang y similares), la biblioteca estándar del lenguaje elegido y el cliente de base de datos que ese lenguaje requiera para comunicarse con PostgreSQL. Docker, Compose, Kubernetes y Kustomize son las herramientas de trabajo; lo que se restringe es la delegación de la configuración que esta tarea evalúa. No se requiere implementar mecanismos de bajo nivel como *namespaces* o *cgroups*.

La frontera se aplica también a la autenticación, pero en sentido inverso. Aquí lo correcto es *no* escribir el mecanismo a mano: no se acepta implementar el inicio de sesión, la emisión ni la firma de tokens por cuenta propia. La autenticación se resuelve integrando el proveedor externo (Keycloak), y el trabajo del grupo consiste en configurarlo y en validar contra él los tokens que recibe. Las bibliotecas de cliente OIDC o de validación de JWT del lenguaje elegido se usan sin restricción.

4 Módulo adicional para grupos de tres

Los grupos de tres personas trasladan el mismo sistema a Kubernetes sobre un clúster local levantado con kind. Se trata del mismo sistema expresado en otro modelo de orquestación, y el ejercicio consiste en establecer la correspondencia entre cada concepto de Compose y su objeto equivalente en Kubernetes, así como en identificar aquellos que carecen de equivalente directo.

Se requieren los manifiestos completos del sistema, que comprenden un Deployment para el servicio y otro para PostgreSQL, los Service que los exponen dentro del clúster (con el que permite alcanzar la aplicación desde la máquina anfitriona), un PersistentVolumeClaim que asuma el papel del volumen de Compose, y la configuración externalizada mediante ConfigMap y Secret. Sobre esa base se solicita una personalización con Kustomize, con una carpeta base/ de manifiestos comunes y al menos un overlay/ que modifique algo real (el número de réplicas, el tag de la imagen o los valores de configuración) sin duplicar los manifiestos originales. Las comprobaciones de *liveness* y *readiness* del servicio se declaran como livenessProbe y readinessProbe del Deployment, en correspondencia con las rutas /health y /ready. El README indica el comando exacto para crear el clúster, aplicar el overlay y verificar que el sistema responde.

Este módulo **no otorga puntos extra**, ya que constituye el alcance esperado de un grupo de tres personas. Un grupo de tres que entregue solo el núcleo se evalúa con el módulo de Kubernetes en cero.

5 Entrega

La tarea se publica en la Semana 1 y se entrega en la Semana 3, a más tardar a las 10:00 p.m. por el buzón de TecDigital; la fecha oficial es la publicada en Discord. Cada día de atraso descuenta cinco puntos sobre base cien. Esta tarea no contempla defensa, de modo que la evaluación se realiza exclusivamente sobre lo que el repositorio y el video demuestren.

La entrega en TecDigital consiste en un único paquete que contiene:

1. El enlace al repositorio de GitHub. El historial debe mostrar *commits* de todos los integrantes; la autoría individual se determina por los *commits* y, en los archivos comunes, se considera compartida. Un integrante sin *commits* se evalúa individualmente según la autoría que pueda demostrarse.
2. El código fuente completo del servicio, las pruebas unitarias y de integración, el `Dockerfile`, el `docker-compose.yml`, el `.env.example`, el `.dockerignore` y el `.gitignore`, sin dependencias instaladas, de modo que el repositorio no contenga `node_modules`, `.venv`, `vendor` ni equivalentes. Los grupos de tres incluyen además la carpeta de manifiestos de Kubernetes con su `base/` y su `overlay/`.
3. La documentación, en formato **Markdown** dentro del repositorio, que comprende el `README.md` con las instrucciones reproducibles, el contrato de las rutas y las decisiones técnicas. No se aceptan documentos en Word ni PDF como sustituto.
4. Un video de demostración de máximo diez minutos, **sin edición**, que muestre el flujo completo, es decir, la clonación del repositorio, el arranque del sistema con un solo comando, el ejercicio de las rutas del servicio, la comprobación de la persistencia tras un reinicio y, en los grupos de tres, el despliegue en el clúster de `k8s` con el `overlay` aplicado. El video tiene un fin práctico, ya que evita que la evaluación se bloquee por un problema de entorno en la máquina del evaluador.

6 Evaluación

La evaluación distingue el *criterio técnico demostrable* (70 puntos), que recoge lo que puede verse funcionando con evidencia reproducible, y la *profesionalidad* de la entrega (30 puntos). Cada criterio tiene un peso, y la nota de cada uno resulta de multiplicar ese peso por el nivel de logro alcanzado según la matriz. En una tarea corta no se exige documentación a profundidad, pero sí que lo entregado esté ordenado, se entienda y corra.

Rúbrica de la Tarea Corta 1

Criterio (peso)	Excelente (100 %)	Bueno (75 %)	Regular (50 %)	Deficiente (≤25 %)
Dockerfile propio (10)	Imagen base oficial justificada, dependencias mínimas y sin artefactos innecesarios; puerto y arranque correctos.	Funciona, con alguna dependencia o archivo de más, o justificación incompleta.	Construye, pero con imagen inflada o decisiones sin justificar.	No construye, o parte de una imagen que ya hace el trabajo del servicio.
Sistema funcional end-to-end (12)	Un solo comando levanta todo; la aplicación alcanza la base por nombre de servicio; el recurso recorre su ciclo CRUD y las rutas de liveness y readiness cumplen su contrato con los códigos de estado correctos.	Levanta y las rutas responden, con algún código incorrecto o acceso por IP o localhost.	Levanta con pasos manuales, o alguna ruta no cumple el contrato.	No levanta con un comando, o las rutas no funcionan.
Autenticación con Keycloak (10)	Keycloak configurado (realm y client); las rutas de modificación exigen token válido, con 401 sin token y 403 sin el rol, y la firma se valida contra el proveedor.	Protección funcional, con un detalle incompleto (sin verificación de rol o validación laxa).	Protección parcial, o el token no se valida correctamente.	Sin autenticación, o implementada a mano en vez de con Keycloak.
Pruebas unitarias y de integración (10)	Ambos tipos corren con un comando y cubren el CRUD, la validación y la autenticación; todas pasan.	Pruebas de ambos tipos, con cobertura parcial.	Solo unitarias, o pruebas superficiales.	Sin pruebas válidas.
Persistencia (8)	Volumen declarado; los datos sobreviven a down y up de forma verificable.	Volumen presente, con verificación incompleta.	Persistencia frágil o dependiente de pasos manuales.	Sin volumen; los datos se pierden al recrear los contenedores.
Healthchecks y arranque (6)	Healthchecks declarados y dependencia de arranque resuelta y justificada.	Healthchecks presentes y dependencia resuelta, pero sin justificar.	Un solo healthcheck, o dependencia resuelta de forma frágil.	Sin healthchecks; la aplicación arranca antes que la base.
Configuración sin secretos (6)	Variables de entorno, .env.example versionado y secretos fuera de la imagen y del repositorio.	Configuración externalizada con un descuido menor.	Algún valor sensible en el código o en el Compose.	Credenciales dentro de la imagen o del repositorio.
Kubernetes y Kustomize (8, solo grupos de 3)	Deployment, Service y PVC funcionales en kind; base y overlay sin duplicar manifiestos.	Manifiestos funcionales, con overlay que duplica algo o configuración incompleta.	Despliegue parcial o sin Kustomize.	No se implementó.
README reproducible (14)	Markdown claro con requisitos, arranque, contrato de las rutas, ejecución de pruebas, verificación de persistencia y apagado; un tercero lo sigue sin ayuda.	Cubre lo esencial, con alguna omisión.	Instrucciones incompletas o confusas.	Ausente o insuficiente para ejecutar.
Repositorio y autoría (8)	Historial con commits de todos, estructura ordenada y sin dependencias versionadas.	Commits de todos, con algún descuido de orden.	Contribución desigual o dependencias versionadas.	Un solo autor, o repositorio desordenado.
Video y entrega (8)	Video de máximo diez minutos sin edición que muestra el flujo completo; entrega conforme.	Video que cubre casi todo el flujo.	Video parcial, o incumple algún requisito de entrega.	Sin video, o no demuestra el sistema.

Grupos de dos. El criterio de Kubernetes y Kustomize no aplica y sus 8 puntos se redistribuyen en el bloque técnico, con lo que Dockerfile propio pasa a 12, sistema funcional a 14, y autenticación y pruebas a 12 cada una; persistencia, healthchecks y configuración sin secretos se mantienen en 8, 6 y 6. Un grupo de tres que no entregue ese módulo obtiene el nivel Deficiente en él.

Uso de inteligencia artificial. El uso de herramientas de IA está permitido, pero cada entrega incluye una breve declaración de cómo se usaron, en qué partes y qué se verificó de forma manual. Declararlo no penaliza; ocultarlo agrava. Si en la revisión no se distingue la aportación intelectual del grupo, porque el trabajo aparente haber sido producido de forma exclusiva por una herramienta automática, la entrega queda sujeta a penalización, según el Principio de Aportación Demostrable y la cláusula BDFL de las disposiciones del curso.

Aplican además las penalizaciones transversales del curso. Se descuentan cinco puntos por cada día de atraso sobre base cien. La ausencia de *commits* de un integrante convierte su evaluación en individual, según la autoría que pueda demostrarse. Subir al repositorio las carpetas de dependencias ya instaladas (*node_modules*, *.venv*, *vendor* y equivalentes), en lugar de excluirlas con *.gitignore*, descuenta cinco puntos del bloque de profesionalidad.